GraphQL Shield

Dashboard verification guide: what every section does, the attacks it shows, and how to check each one step by step

Who this is for: anyone who wants to check the dashboard with their own eyes: you, your team, your guide or an examiner. No coding knowledge is needed except for the few commands in Part 2 and Part 7.

How to use it: do Part 2 once to start the project. Then go through Parts 3 to 6 in order. Each check has numbered steps and a green box saying what you should see. Part 8 is a one-page checklist to tick off.

Contents

- Part 1. What the project does, in simple words
- Part 2. Start the project
- Part 3. The attack types (threats) you will test
- Part 4. Tab 1: Security Testing
- Part 5. Tab 2: Research & ML Evaluation
- Part 6. Tab 3: Audit & Security Verification
- Part 7. Account protection and command-line checks
- Part 8. Final checklist and troubleshooting

Part 1. What the project does, in simple words

Many apps use **GraphQL**: one web address that answers many kinds of questions, such as "give me document 102" or "give me user 101 and their documents". The danger is that a logged-in user asks for an object that belongs to **someone else**, simply by changing the ID number. This attack is called **BOLA** (Broken Object Level Authorization). It is the number one risk in the OWASP API Security Top 10.

GraphQL Shield sits in front of the app and makes sure every user can only see and change **their own** objects. It does this with two layers of checks:

Part	What it does	Simple picture
Layer 1 (before the query runs)	Reads the query, finds every ID in it, and checks who owns each one. If any ID belongs to someone else, the whole request is refused before anything runs.	A security guard checking your ticket at the door.
Layer 2 (while the query runs)	Checks every object the app is about to send back, and every object a change (mutation) would touch. Foreign objects are removed or the change is refused.	A second guard inside who checks each item you carry out.
Audit log	Every decision (allowed or denied, by which layer, and why) is written to a tamper-evident log.	The CCTV recording.
Blind spot	An object Layer 2 had to stop that Layer 1 never saw. It is proof that Layer 1 missed something.	Someone who got past the door guard but was caught inside.
ML and research part	Uses the blind-spot record to predict, measure and even repair what Layer 1 misses. Never changes an allow or deny decision.	A report that tells you where the door guard is weak, and how to fix it.
Account protection	Locks sign-in after too many wrong passwords, ends sessions on logout, enforces strong passwords.	Stops the attacker getting in as someone else in the first place.

There are **two apps** inside the project, both protected the same way:

- **Demo app** (`/graphql`): users, documents and folders.
- **Clinic app** (`/clinic/graphql`): doctors, patients, medical records and prescriptions. It proves the protection works on a second, different app.
- Each app also has an **insecure baseline** (`/graphql-insecure`, `/clinic/graphql-insecure`): the same app with no protection, on purpose, so you can see the attack succeed there and fail on the protected one.

The demo accounts

Every password follows one pattern: **username + 123**.

User ID	Name	Username	Password	Role	What they own
101	B.V. Balanilavan	balanilavan	balanilavan123	user	Document 101, folder 201 (with documents 202, 203)
102	Kapilesh N	kapilesh	kapilesh123	user	Document 102, and 204 (filed inside Balanilavan's folder 201)
103	Sakthivel B	sakthivel	sakthivel123	user	Document 103
104	Admin Demo Account	chinmayanand	chinmayanand123	admin	Document 104. Needed for the admin panels.

Most tests below sign in as **balanilavan** and try to read **Kapilesh's** document **102** or **204**. Those attempts must be blocked.

Clinic app data: patients 1, 2, 3 belong to doctors 101, 102, 103. Records: 11 (doctor 101), 12 (doctor 102, on patient 1), 21 (doctor 102), 31 (doctor 103).

Part 2. Start the project

2.1 One-time settings

1. Open the project folder `C:\Users\CHINMAYANAND\Downloads\GraphQL-Shield` in VS Code.
2. Open the file `.env`. Check these two lines say `true`. They turn on the insecure baseline and the layer on/off switches, which the demos need: `ENABLE_INSECURE_BASELINE=true` and `ENABLE_LAYER_TOGGLE=true`.
3. Save the file.

> **Be careful**
> These two switches are **for local demos only**. In a real deployment they must be `false`: the insecure endpoint is deliberately vulnerable, and the layer switch affects every user of the server.

2.2 Reset the demo data (do this before a demo)

Open a terminal in the project folder and run:

```
python -m app.seed --with-shared-folder
```

> **What you should see**
> The terminal prints `Database successfully seeded.` All demo accounts and documents are back to their starting state, including document 204 inside folder 201.

2.3 Start the server

```
uvicorn app.main:app --port 8000
```

> **What you should see**
> The terminal prints `Uvicorn running on http://127.0.0.1:8000`. Leave this terminal open while you use the dashboard.

2.4 Open the dashboard

1. Open a web browser (Chrome or Edge).
2. Go to **http://127.0.0.1:8000/dashboard**
3. At the top right you should see **Connected** (not "Connecting to API…"), and the left side says **Protection enabled**.

The dashboard has **three tabs** at the top: **Security Testing**, **Research & ML Evaluation**, and **Audit & Security Verification**. The buttons in the top bar do the following:

Button	What it does
☼ Theme	Switches between light and dark colours.

Button	What it does
⏸ Pause auto-refresh	Stops the numbers updating every few seconds, so you can read or take a screenshot.
↻ Refresh	Reloads all numbers now.
⇩ Export CSV	Downloads the audit records as a spreadsheet file.

Part 3. The attack types (threats) you will test

The test console on Tab 1 has seven scenarios. Each one is a different trick an attacker uses to reach someone else's data. Read this table once; Part 4 shows how to run each one.

#	Scenario in the console	What the attacker tries	Why it is dangerous	Which layer stops it
1	Direct resource lookup	Asks directly for document 102: `resource(id: 102)`.	The simplest BOLA: change the number in the request.	Layer 1 (before execution)
2	Nested user → resource lookup	Hides 102 inside a query about their own user: `user(id: 101) { resource(id: 102) }`.	Simple checkers only look at the top level and miss nested IDs.	Layer 1
3	Implicit children relationship	Lists their own folder 201, which also holds Kapilesh's 204: `resource(id: 201) { children { … } }`.	No foreign ID appears in the request at all, so a before-execution check cannot see it. This is the main reason Layer 2 exists.	Layer 2 (removes 204 from the answer)
4	Inline fragment user lookup	Wraps the lookup in GraphQL fragment syntax: `... on Query { user(id: 102) }`.	Tries to slip past a checker that does not understand fragments (a Layer 1 bypass attempt).	Layer 1, and Layer 2 as a backstop
5	Batched request (2 operations)	Sends a harmless query and an attack together in one request.	Hopes the harmless one makes the batch look safe.	Layer 1 (the whole batch is refused)
6	Mutation: rename resource	Tries to rename Kapilesh's document 102.	A write attack: changing someone else's data.	Layer 1 and Layer 2 (refused before the change runs)
7	Mutation: delete resource	Tries to delete Kapilesh's document 102.	The most damaging write attack.	Layer 1 and Layer 2

Other threats the project covers

Threat	What it means	Where to check it
Information leak: foreign vs missing	If "not yours" and "does not exist" gave different answers, an attacker could learn which IDs exist.	Part 6.4 (Leak Channels card) and Part 5.6 (timing channel).
Timing leak	Even with identical answers, a slower reply for "not yours" would leak the same fact.	Part 5.6, Timing channel panel.

Threat	What it means	Where to check it
Password guessing	Trying many passwords for one account.	Part 7.1: lockout after 5 wrong passwords.
Stolen session token	Using someone's login token after they log out.	Part 7.2: logout revokes the token.
Configuration mistake	A developer forgets to register a type or argument, so Layer 1 stops checking it.	Part 5.5: self-repairing configuration.

Part 4. Tab 1: Security Testing

Click **Security Testing** at the top. This is where you run attacks yourself and watch them get blocked.

4.1 The five number cards at the top

What it is: live totals from the audit log.

Card	Meaning
Total requests	All requests recorded in the audit log.
Allowed	Requests that were allowed.
Blocked	Requests that were denied.
BOLA signals	Ownership violations caught, including nested and relationship ones.
High risk	Audit entries marked high risk (repeated or suspicious attempts).

How to verify:

1. Write down the current **Blocked** number.
2. Run any attack from 4.3 below.
3. Wait 2 to 3 seconds, or click ↻ **Refresh**.

> **What you should see**
> **Total requests** and **Blocked** both go up. The cards are reading the real log, not fixed numbers.

4.2 Demo users and roles

What it is: the four local test accounts (table in Part 1).

1. Find the table **Local demo accounts**.
2. Click the action button next to **balanilavan**.

> **What you should see**
> The **Login username** box in the test console below is filled with `balanilavan`. You still type the password yourself; the page never stores it.

4.3 GraphQL test console: run each attack

What it is: a small tool that sends real GraphQL requests to the server and shows the answer.

First, sign in

1. In **Login username** type `balanilavan`, in **Login password** type `balanilavan123`.
2. Click ▶ **Run selected test** once (the first run signs you in).

> **What you should see**
> The status line says you are logged in as Balanilavan (user 101). It stays logged in for this browser tab.

Attack 1: Direct lookup

1. **Endpoint:** Protected GraphQL Shield. **Scenario:** Direct resource lookup.
2. **Resource ID:** `101`. Click ▶ **Run selected test**.
3. Now change **Resource ID** to `102`. Click ▶ **Run selected test** again.

> **What you should see**
>
> With 101 (your own) the answer shows the document's name. With 102 (Kapilesh's) the request is **denied**: an error such as "not authorized" and no data from document 102.

Attack 2: Nested lookup

1. **Scenario:** Nested user → resource lookup. **User ID:** `101`. **Resource ID:** `102`.
2. Click ▶ **Run selected test**.

> **What you should see**
>
> Denied. Layer 1 found the hidden ID 102 inside the nested query.

Attack 3: Children relationship (the important one)

1. **Scenario:** Implicit children relationship. **Parent/folder ID:** `201`.
2. Click ▶ **Run selected test**.

> **What you should see**
>
> The request **succeeds** and shows folder 201 with documents **202 and 203** only. Kapilesh's **204 is missing** from the list. The request itself was fine (it is your folder), so it is not refused; only the foreign item is removed. This is Layer 2 at work.

Attack 4: Inline fragment

1. **Scenario:** Inline fragment user lookup. Run it with the user ID set to `102`.

> **What you should see**
>
> Denied: you cannot read user 102's details through a fragment.

Attack 5: Batch

1. **Scenario:** Batched request (2 operations). Keep Resource ID `102`. Run it.

> **What you should see**
>
> The whole batch is refused. The harmless half does not make the attack go through.

Attacks 6 and 7: Rename and delete

1. **Scenario:** Mutation: rename resource. **Resource ID:** `102`. Run it.
2. **Scenario:** Mutation: delete resource. **Resource ID:** `102`. Run it.

3. Switch back to **Direct resource lookup**, sign in as `kapilesh` / `kapilesh123`, and read `102`.

> **What you should see**
> Both changes are refused. When Kapilesh reads 102, its name is **unchanged** and it still exists.

Same attacks on the clinic app (second application)

1. **Endpoint:** Clinic app, protected. Sign in as `balanilavan` again.
2. **Scenario:** Direct lookup, **Resource ID:** `1` (your patient), then `2` (doctor 102's patient).

> **What you should see**
> Patient 1 is shown, patient 2 is denied. The same protection works on a completely different app.

4.4 Compare protected vs insecure

What it is: sends the same request to the protected app and to the insecure baseline, and shows both answers side by side.

1. Sign in as `balanilavan`. **Scenario:** Direct resource lookup. **Resource ID:** `102`.
2. Click ⇄ **Compare protected vs insecure**.

> **What you should see**
> **Protected endpoint:** denied. **Insecure baseline:** returns Kapilesh's document 102 ("Kapilesh's Financial Record"). This proves the attack is real, and that the Shield is what stops it.

> **Why this matters**
> Without the comparison, someone could say "the attack never worked anyway". The insecure side shows that it does work when the Shield is absent.

4.5 Run all tests

What it is: runs eight standard checks in one click and gives a verdict for each.

1. Sign in as `balanilavan`.
2. Click ✓ **Run all tests**.

Test	What it tries	Expected on protected
Own document	Read 101 (yours)	Allowed
Direct attack	Read 102	Blocked
Nested attack	Hide 102 inside a user query	Blocked
Inline fragment	Look up user 102 inside a fragment	Blocked
Folder children	List folder 201, which holds Kapilesh's 204	Only your documents
Batch	Harmless query plus an attack	Blocked
Rename attack	Rename Kapilesh's 102	Blocked, unchanged

Test	What it tries	Expected on protected
Delete attack	Delete Kapilesh's 102	Blocked, not deleted

> **What you should see**
>
> Every row's **Verdict** is a pass (green). The **Insecure baseline** column shows the read attacks **leaking** there. Mutations are skipped on the insecure baseline, because they would really change data.

4.6 Per-request taint → authorization flow

What it is: an explanation of the last request you ran: where each ID came from (the "taint source", for example a variable you typed), which object path it reached, who owns that object, and why the policy allowed or denied it.

1. Run **Attack 2 (Nested lookup)** from 4.3.
2. Scroll to **Per-request taint → authorization flow**.

> **What you should see**
>
> Numbered boxes show: 1 the request input with ID 102, the path `user → resource`, the owner (Kapilesh, 102), the caller (101), and the decision **DENY** with the reason. A note reminds you that a tainted input alone is not an attack; the ownership check decides.

4.7 Dual-layer defense (admin)

What it is: live counters per layer, the switches to turn each layer off for a demo, and a table of layer events.

Counter	Meaning
Layer 1 blocked	Requests refused before execution.
Layer 2 filtered	Foreign objects removed from answers at runtime.
Layer 2 mutations blocked	Changes stopped before they ran.
Layer 1 blind spots	Objects only Layer 2 caught: proof that Layer 1 alone is not enough.

Show why both layers are needed

1. In **Admin username / password** type `chinmayanand` / `chinmayanand123`, click **Sign in as admin**.
2. Click **Turn Layer 2 OFF** and confirm the warning. A red banner appears: "Layer 2 is switched OFF. Protection is reduced for every user of this server until it is turned back on."
3. Go back to the console, sign in as `balanilavan`, and run **Attack 3 (children of folder 201)**.
4. Click **Turn both layers back on** in the red banner.
5. Run Attack 3 again.

What you should see

With Layer 2 off, the answer **includes Kapilesh's 204**: Layer 1 alone leaks it. With both layers on, 204 is gone again and **Layer 1 blind spots** goes up by one.

Be careful

Always turn both layers back on after this demo. The switch applies to every user of the server.

4.8 Where taint analysis fits

What it is: a five-step explainer (taint sources → propagation → relationship graph → authorization reasoning → runtime enforcement). It is for reading only; nothing to click. It explains that the analysis finds suspicious inputs, but only the ownership check decides allow or deny.

Part 5. Tab 2: Research & ML Evaluation

Click **Research & ML Evaluation**. This tab holds the machine learning models and the experiment results from the paper. Important rule: **the ML never decides allow or deny.** It only observes, predicts and reports.

5.1 Behavioural ML model (advisory)

What it is: an XGBoost model and a simple hand-written rule each give every user's recent activity a "suspicious" score (for example, very fast requests, many denials, trying many other people's IDs). They were tested on a type of session ("nested relational") the model never saw in training.

1. Read **Model loading status**: it should say the model is loaded.
2. Read the **Benchmark metric comparison** table.

What you should see

XGBoost accuracy **99.2%**, rule baseline **100%**. The badge **RULE BASELINE OUTPERFORMED ML** and the notice **No ML superiority claim** are shown.

Why this matters

This is honest reporting: the project does not pretend ML is better when a simple rule does as well. Reviewers value this.

5.2 Suspicious sessions and telemetry alerts (admin)

1. Make sure you are signed in as admin (Part 4.7).
2. Run a few attacks as `balanilavan` in Tab 1, for example click **Run all tests** twice.
3. Come back and look at **Suspicious behavioural sessions**.

What you should see

A row for user 101 with a higher alert level, a high **Denials** and **Cross-owner** count, and scores from both XGBoost and the rule. If there is no recent traffic, labelled sample sessions are shown instead.

5.3 Interactive session telemetry scorer

What it is: lets you type in made-up session numbers and see how both models score them.

1. Under **Load scenario preset** click **Normal steady access**, then ⚡ **Score telemetry**.
2. Click **Rapid enumeration attack**, then ⚡ **Score telemetry**.
3. Try **Suspicious nested probing** and **Legitimate high activity** too.

What you should see

Normal steady access → **NORMAL / Benign**, probability near 0. Rapid enumeration → **suspicious**, probability near 1. Legitimate high activity stays benign even though it is busy: speed alone is not treated as an attack.

5.4 Layer 1 blind-spot predictor

What it is: a model that looks at a query **before it runs** and predicts whether it will reach objects Layer 1 cannot see (a blind spot, like the children attack). Its training labels come from Layer 2's audit record, so nobody had to label data by hand.

1. Read the **evaluation split** table (F1, PR-AUC, recall for XGBoost and for the rule).
2. Look at **What drives the prediction**: the features that matter most (for example, relationship fields such as `children`).
3. Signed in as admin, the box **Score a query without running it** holds `{ resource(id: 201) { id children { id name } } }`. Click **Predict blind spot**.
4. Replace it with `{ resource(id: 101) { id name } }` and predict again.
5. Look at **On live traffic**.

> **What you should see**
>
> The children query is predicted as a **blind spot**; the simple lookup is not. **On live traffic** shows how many real blind-spot denials were flagged before they ran.

5.5 Self-repairing configuration (admin)

What it is: the newest feature. When a developer makes a configuration mistake (for example forgets to protect a type), Layer 1 misses objects and Layer 2 records them. The project reads that record, works out **which configuration line is wrong**, proposes a fix, **tests the fix offline** by replaying recorded requests, and only then shows it. Only an admin can apply it, and it can be undone.

Live demo, step by step

1. Signed in as admin, scroll to **Self-repairing configuration**.
2. In the drop-down pick a mistake, for example **demo: missing_resource_type** (the developer forgot to protect documents).
3. Click **Introduce this mistake**.
4. Go to Tab 1, sign in as `balanilavan`, and click **Run all tests**. The direct and nested attacks on document 102 now succeed and show Kapilesh's data: with ResourceType unprotected, neither layer checks who owns a document. This is the leak the repair has to close.
5. Come back to Tab 2 and click **↻ Analyse now**.
6. Read the repair card: what it changes ("add ResourceType to owned types"), the evidence it is based on, and the verification result (gaps closed, legitimate requests blocked: 0).
7. Click **Apply repair**.
8. Run **Run all tests** again in Tab 1.
9. In **Applied repairs** you see the repair with a **Revert** button. Finally click **Restore original configuration**.

What you should see

After step 6 exactly one verified repair is offered, and it is the correct one. After applying it, Layer 1 blocks the attacks again. After restoring, the configuration is back to normal.

The evaluation table

What you should see

30 / 30 mistakes fully restored by the first verified repair; **33 / 33** wrong (decoy) repairs refused; **0** repairs proposed when the configuration was correct.

Why this matters

Most real security bugs are configuration mistakes, not code bugs. This shows the system can find its own weak point and fix it safely, with a human in control.

5.6 Research evidence panels

What it is: results of the paper's experiments, read from `app/models/research_results.json`. They are fixed numbers from the experiment scripts, not live. To rebuild them: `python -m scripts.build_dashboard_results`.

Panel	What it shows	What to check
Layer ablation	The same six attacks with each layer on or off.	Only **both layers on** stops all six. Layer 1 alone leaks the relationship (children) attack.
Comparison with graphql-shield (Node.js)	Our system vs the popular graphql-shield library on the same 13 attacks and 5 normal requests.	Our system: **0 attacks leaked, 0 normal requests failed**. Check that the other rows leak or fail some.
Latency per protected request	How much time protection adds per request, with a 95% confidence interval.	Overhead is about **1 ms** or less.
Blind-spot predictor across random seeds	The predictor repeated on fresh data.	Model and simple rule are equally accurate (honest result).
Fault injection	17 realistic bugs planted in Layer 1, 5 runs each. Did the evidence detect them?	**17 / 17 bugs detected**, **0 false alarms** on the unmodified Layer 1. The per-bug table lists each one.
Latency as responses grow	A folder with up to 200 children, checked one by one vs batched.	Batched is faster (about 26 ms vs 45 ms at 200 children).
Timing channel	Whether "not yours" takes longer to deny than "does not exist".	Gap close to 0 ms and **p above 0.05**: no measurable timing leak after the fix.

Part 6. Tab 3: Audit & Security Verification

Click **Audit & Security Verification**. This tab proves what happened, from the log and from the automated tests.

6.1 Request decision mix

What it is: a ring chart of allowed vs denied decisions.

> **What you should see**
>
> The **Allowed** and **Denied** numbers add up to the decisions total, and they match the cards on Tab 1. The note says this is a count of decisions, not an accuracy score.

6.2 Authorization overview

What it is: bars comparing allowed and denied decisions and high-risk records vs total requests, with a short risk summary.

> **What you should see**
>
> The bars change after you run attacks in Tab 1 and click ↻ **Refresh**.

6.3 Decisions over time

1. Choose **Last 24 hours**, then **Last 14 days**, then **Last 90 days**.

> **What you should see**
>
> A chart of allowed and denied decisions per hour or per day (UTC). Your test runs from today appear as a spike at the right end.

6.4 Verified security property tests

What it is: 12 cards showing the result of the last automated test run for each security property in the paper. These are not typed in by hand; they are read from the last `pytest` run.

Card	What the property means
P1 Safety	No answer ever contains an object the caller does not own.
P2 Non-interference	Changing only other users' objects never changes your answer (so you cannot learn about them).
P3 Write safety	No request can change an object the caller does not own.
P4 Availability	Requests for your own objects still work (protection does not break normal use).
Properties fail without protection	With both layers off, P1 to P3 fail. This proves the tests can actually catch a leak.
Leak channels closed	Same reply for foreign and missing IDs, for interfaces, counts and timing.

Card	What the property means
Other six cards	Single access, nested access, insecure endpoint isolation, confidentiality, mutation independence, derived values (counts).

1. Stop the server (Ctrl + C), run `python -m pytest -q` (Part 7.3), then start the server again.
2. Open this tab and click ↻ **Refresh**.

> **What you should see**
> All 12 cards show **passed** with the number of tests, and the date of the run you just did.

6.5 Recent authorization decisions and runtime provenance

What it is: the latest Layer 2 decisions, with the object, its owner, the path in the answer where it appeared (for example `resource.children`) and its parent object.

1. Run **Attack 3 (children of folder 201)** in Tab 1 as `balanilavan`.
2. Come back and refresh.

> **What you should see**
> A row with actor **101**, object **204**, owner **102**, path `resource.children`, parent **201**, decision **DENY**.

6.6 Recent audit events (search and filter)

1. Set **decision** to **Denied** and **user** to **101 · Balanilavan**.
2. Set **layer** to **Layer 1**, then to **Layer 2**, and compare.
3. Click ⇩ **Export visible** to download what you see.
4. Click **Clear filters**. Use **Load 25 more** to see older entries.

> **What you should see**
> Only matching rows are shown. Times are shown in IST. At the bottom, the footer should say ✓ **Audit log intact: … chained rows verified**. Each record is chained to the previous one, so editing old records would be detected.

Part 7. Account protection and command-line checks

7.1 Password guessing is locked out

1. In the Tab 1 console, type username `sakthivel` and a wrong password, for example `wrong`. Click ▶ **Run selected test**. Do this **5 times**.
2. Now type the **correct** password `sakthivel123` and try again.

> **What you should see**
> After 5 wrong tries the login is refused with "Too many failed sign-ins: this account or address is locked for about 15 minutes" (HTTP 429), **even with the correct password**. The attacker cannot keep guessing.

To unlock it for the next demo, stop the server and run:

```
python -m scripts.unlock_sign_in sakthivel
```

7.2 Logout really ends the session

1. Sign in as `balanilavan` in the console and run any test.
2. Click **Log out** in the console.

> **What you should see**
> You are signed out, and the old token is **revoked on the server**. Anyone who copied it gets "Session ended. Please sign in again." (HTTP 401). This is covered automatically by `tests/test_account_security.py`.

7.3 Run all automated tests

```
python -m pytest -q
```

> **What you should see**
> About **421 passed, 6 skipped** (4 Graphene and 2 Ariadne tests need optional libraries; with Ariadne installed it is 423 passed, 4 skipped). **0 failed.** 427 tests in all.

7.4 Re-run the experiments (optional)

Command	What it re-checks
`python -m scripts.run_repair_evaluation`	Self-repair: 10 mistakes, 3 seeds (Part 5.5).
`python -m scripts.run_timing_channel`	Timing channel (Part 5.6).
`python -m scripts.run_latency_benchmark`	Latency overhead (Part 5.6).
`python -m scripts.run_scaling_benchmark`	Latency as responses grow (Part 5.6).

Command	What it re-checks
`python -m scripts.build_dashboard_results`	Copies all results into the dashboard. Run it last, then refresh the page.

Part 8. Final checklist and troubleshooting

8.1 Checklist

✓	Check	Expected
☐	Dashboard opens, says Connected	Part 2.4
☐	Own document 101 readable	Allowed
☐	Direct, nested, fragment, batch attacks on 102	All blocked
☐	Folder 201 children	202 and 203 only, 204 removed
☐	Rename and delete 102	Blocked, 102 unchanged
☐	Clinic app patient 2 as balanilavan	Blocked
☐	Compare protected vs insecure	Insecure leaks, protected blocks
☐	Run all tests	All 8 verdicts pass
☐	Layer 2 off → children leak; back on → fixed	Blind spots counter goes up
☐	Behavioural model and scorer presets	Enumeration flagged, normal benign
☐	Blind-spot predictor on the children query	Predicted blind spot
☐	Self-repair demo	One verified, correct repair; applied; restored
☐	Research evidence panels	Numbers as in Part 5.6
☐	12 property cards	All passed
☐	Provenance row for object 204	DENY, path resource.children
☐	Audit filters and integrity	Log intact
☐	5 wrong passwords	Locked, even with correct password
☐	Logout	Old token refused
☐	pytest	0 failed

8.2 Troubleshooting

Problem	Fix
Dashboard says "Connecting to API…"	The server is not running. Start it (Part 2.3) and refresh.
Login refused: too many attempts	Run `python -m scripts.unlock_sign_in --all`, then try again.
Insecure baseline does not answer	Set `ENABLE_INSECURE_BASELINE=true` in `.env` and restart the server.
Turn Layer 1/2 OFF buttons or Introduce this mistake do nothing	Set `ENABLE_LAYER_TOGGLE=true` in `.env`, restart, and sign in as admin.

Problem	Fix
Document 204 is missing everywhere, even on the insecure side	Reset the data: `python -m app.seed --with-shared-folder`.
Document 102 was changed or deleted	Only possible on the insecure baseline. Reset the data as above.
Property cards say no test run found	Run `python -m pytest -q`, then refresh.
Research panels say Loading… for a long time	Run `python -m scripts.build_dashboard_results`, then refresh.
A red banner says a layer is switched off	Click **Turn both layers back on**.

Golden rule: everything in this guide runs only on your own computer, against your own local demo data. Never point the attack scenarios at a server you do not own.